Tradução PT-BR — Capítulo 2

Arquivo original: TG2_Bruno_Silveira.pdf

Capítulo traduzido: 2 Background (Fundamentação Teórica), Seções 2.1 a 2.6.2.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo discute e explica alguns dos conceitos-chave relevantes para compreender este trabalho. As Seções 2.1, 2.2 e 2.3 abordam tópicos relacionados a como Redes Neurais funcionam sob a perspectiva de TinyML: definição de Redes Neurais, operações de multiplicação e acumulação, e quantização. A Seção 2.4 explica brevemente o que é Busca de Arquiteturas Neurais e suas particularidades ao trabalhar com dispositivos pequenos. Por fim, as Seções 2.5 e 2.6 tratam da parte de software do TinyML, respectivamente sobre padrões de benchmarking e sobre frameworks de TinyML.

2.1 Redes Neurais Artificiais

Uma Rede Neural Artificial é um método computacional para resolver problemas inspirado no cérebro humano. A ideia central por trás de Redes Neurais é combinar muitas unidades pequenas de computação simples, chamadas neurônios, de forma estruturada, em que as saídas de uma camada são usadas como entradas da próxima, de modo que, em conjunto, elas realizem computações complexas. Em particular, redes com muitas camadas (com múltiplas camadas ocultas) são conhecidas como Redes Neurais Profundas. DNNs são especialmente boas em tarefas de classificação, razão pela qual, nos últimos anos, sua aplicação cresceu de forma intensa em diversos campos, como reconhecimento de imagens e de fala (SZE et al., 2017).

O elemento-chave em uma rede neural é o neurônio, e é essencial compreender como ele opera. Um neurônio recebe N entradas e possui uma saída, calculada pela Equação 2.1, em que $y \in R$ é a saída, $x \in R^N$ é o vetor de N entradas, $w \in R^N$ é um vetor de N pesos e $b \in R$ é o viés (bias) do neurônio (SZE et al., 2017).

$$y = f(\Sigma_{i=1..N} w_i x_i + b) \quad (2.1)$$

Essa computação de “multiplicar e somar” pode ser vista simplesmente como o produto interno (dot product) dos vetores x e w. Como o produto interno é uma operação linear, adiciona-se a função não linear f para introduzir não linearidade ao sistema. Como a saída de um neurônio também é chamada de ativação, essa função f é chamada de função de ativação (SZE et al., 2017).

Em uma rede neural típica, os neurônios são organizados em camadas, como mostrado na Figura 2.1. Uma camada pode ser entendida como as conexões entre dois grupos de neurônios e sua saída coletiva. Para ilustrar, considere a Figura 2.1. A Camada 1 conecta 3 neurônios de entrada (que não realizam computação, pois são apenas o vetor de entrada da rede) a 4 neurônios intermediários. Como cada par de neurônios possui uma conexão, há um peso w_{ij} associado a cada par. A computação da Equação 2.1 é realizada 4 vezes, cada uma com seu próprio viés e vetor de pesos, mas sobre o mesmo vetor de entrada. Isso significa que há 12 pesos e 4 vieses associados a essa camada, bem como 4 ativações. Esses pesos e vieses são os parâmetros de uma rede. Eles são aprendidos na fase de treinamento e representam uma instância treinada de um modelo (SZE et al., 2017). A partir daqui, os termos modelo e rede serão usados de forma intercambiável para se referir a uma arquitetura de camadas com ou sem parâmetros associados. Uma matriz de parâmetros, como uma matriz de pesos, um vetor de vieses e até um vetor de ativações, também pode ser chamada de tensor.

Figura 2.1: Ilustração de como camadas de rede neural são estruturadas (SZE et al., 2017)

Desenvolver aplicações que utilizam DNNs envolve duas fases principais: treinamento e inferência. A fase de treinamento ocorre quando um conjunto de dados de possíveis vetores de entrada, cada um com seus respectivos rótulos, é alimentado na rede em um processo iterativo, durante o qual os parâmetros da rede

são ajustados seguindo algum algoritmo de otimização, como Stochastic Gradient Descent. Já a fase de inferência ocorre quando o modelo já está treinado e precisa executar suas computações sobre novos dados não vistos, o que acontece continuamente após ser implantado em uma aplicação (SZE et al., 2017). Como este trabalho foca na fase de inferência, técnicas de treinamento estão fora do escopo, e as próximas seções serão restritas principalmente a tópicos que dizem respeito à inferência.

Embora o exemplo da Figura 2.1 mostre todos os pares de neurônios conectados em uma camada, existem outros tipos de camadas (também chamadas operadores) comumente usados em redes neurais. Nesse caso específico, trata-se de uma camada Totalmente Conectada (Fully Connected), que também pode ser representada como a multiplicação matricial mostrada na Equação 2.2, em que $y \in R^M$ é o vetor de M saídas, $x \in R^N$ é o vetor de N entradas, $W \in R^{M \times N}$ é a matriz M×N com os pesos, $b \in R^M$ é o vetor de M vieses, e f é a função de ativação mencionada:

$$y = f(Wx + b) \quad (2.2)$$

Além de camadas totalmente conectadas, há muitos outros operadores usados em redes neurais. Dentre eles, os mais relevantes são (SZE et al., 2017):

• Convolução: realiza uma convolução discreta entre um kernel e os dados. Os pesos do kernel são parâmetros aprendíveis, e múltiplos kernels podem ser convolvidos sobre os mesmos dados, dividindo a rede em diferentes canais paralelos. Cada kernel se estende por toda a profundidade dos canais. O parâmetro stride determina o tamanho do “salto” que o kernel realiza ao convoluir; assim, um stride maior resulta em menor resolução na camada seguinte;

• Convolução em Profundidade (Depthwise Convolution): igual à Convolução, mas cada kernel se aplica apenas a um canal;

• Pooling: reduz o tamanho dos dados agrupando neurônios vizinhos e retornando apenas o maior (MaxPool), o menor (MinPool) ou a média (AvgPool);

• Normalização em Lote (Batch Normalization): normaliza as saídas da camada anterior para que tenham média zero e variância unitária;

• Soma (Add): algumas arquiteturas dividem o caminho em duas rotas, com diferentes camadas em cada uma, chamadas Conexões Residuais (HE et al., 2016). Para recombinar os caminhos, a camada Add é comumente usada, realizando soma elemento a elemento (desde que os tensores tenham o mesmo tamanho).

Por fim, cobrimos as funções de ativação, adicionadas ao final de cada camada para inserir não linearidade. Funções de ativação comuns são (SZE et al., 2017):

• Sigmoid: $y = 1 / (1 + e^{-x})$

• Tangente Hiperbólica: $y = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$

• Unidade Linear Retificada (ReLU): $y = \max(0, x)$

• ReLU com Vazamento (Leaky ReLU): $y = \max(\alpha x, x)$, em que α é uma pequena constante

• Softmax: mapeia valores para uma distribuição de probabilidade válida, com todas as saídas no intervalo [0, 1] e soma igual a 1. Sua fórmula para o i-ésimo elemento de um vetor x é:

$y_i = e^{x_i} / \Sigma_j e^{x_j}$

2.2 Operações Multiply and Accumulate (MACs) em DNNs

O número de operações Multiply and Accumulate (MACs) é uma métrica usada para comparar diferentes modelos em termos de complexidade computacional (TAN et al., 2019). Ela serve ao mesmo propósito que FLOPs (Floating Point Operations), mas não considera cálculos como funções de ativação. Em vez disso, contabiliza apenas as operações de multiplicação e acumulação, que constituem a grande maioria das operações computadas por uma DNN típica. Entre as camadas comumente usadas, o número de MACs é calculado assim para cada uma que realiza essas operações (onde W_i é a largura da entrada da camada, W_o a largura da saída, H_i a altura da entrada, H_o a altura da saída, C_i o número de canais da entrada, C_o o número de canais da saída e K o tamanho do kernel, assumindo kernel quadrado) (GUO et al., 2019):

• Totalmente Conectada (Fully Connected): conforme explicado na seção anterior, a camada é essencialmente uma grande multiplicação de matrizes, onde estão os MACs. Assume-se que entrada e saída são vetores “achatados”, então apenas suas larguras importam no cálculo de MACs, como na Equação 2.3:

$$N_{MACS} = W_i W_o \quad (2.3)$$

• Convolução: para a camada de Convolução, cada pixel em um canal de saída é resultado de todos os MACs entre um kernel e a entrada em todos os canais em um dado passo do kernel (segundo termo da Equação 2.4). Isso é feito para cada pixel de um canal de saída (primeiro termo) e para cada canal de saída (terceiro termo), resultando na Equação 2.4:

$$N_{MACS} = (W_o H_o)(K^2 C_i) C_o \quad (2.4)$$

• Convolução em Profundidade (Depthwise Convolution): é similar à Convolução regular, com a diferença de que, para cada pixel em um canal de saída, o resultado vem de uma multiplicação por kernel 2D, sem o termo de canais de entrada. Assim, o cálculo fica como na Equação 2.5:

$$N_{MACS} = (W_o H_o)(K^2) C_o \quad (2.5)$$

2.3 Quantização

Quantização é uma técnica de compressão de redes neurais em que algumas — ou todas — as partes são convertidas para representações de menor largura de bits, normalmente de ponto flutuante para inteiro. Isso é particularmente útil em TinyML, em que uma pequena redução de acurácia em troca de uma grande redução do tamanho do modelo pode ser desejável, considerando que o tamanho do modelo está entre os principais fatores limitantes para implantar modelos em dispositivos como MCUs (PEREGO et al., 2022).

Há duas alternativas para usar quantização: Treinamento Consciente de Quantização (Quantization-Aware Training) e Quantização Pós-Treinamento (Post-Training Quantization). No primeiro caso, como o nome sugere, a fase de treinamento é alterada ao injetar operações de quantização simuladas e outras técnicas, de modo a otimizar os pesos considerando exatamente como serão quantizados. Isso tende a entregar melhor acurácia e permitir maior compressão quando comparado à Quantização Pós-Treinamento, mas ao custo de exigir um processo de treinamento customizado. No caso comum em que já existe um modelo pré-treinado e não é fácil re-treiná-lo, ou quando é necessária mais flexibilidade durante o treinamento, usa-se a Quantização Pós-Treinamento. Nela, os parâmetros existentes em ponto flutuante são convertidos diretamente para uma representação inteira (ou de ponto flutuante menor). Como este trabalho foca no

caso mais geral de implantar um modelo pré-treinado em um microcontrolador, concentraremo-nos em Quantização Pós-Treinamento (SAHA; SANDHA; SRIVASTAVA, 2022).

Assumindo um modelo com todos os parâmetros e valores intermediários em float32, a quantização inteira é a que mais economiza tempo de computação quando comparada a float16, pois operações inteiras são mais simples e rápidas. A quantização pode chegar a comprimir todos os pesos a um único bit, mas, segundo a revisão de literatura de (SAHA; SANDHA; SRIVASTAVA, 2022), o tipo de quantização mais frequentemente usado em trabalhos recentes de TinyML é int8, que reduz o tamanho de armazenamento em cerca de 4× ao passar de 32 bits para apenas 8 bits para representar parâmetros do modelo. Para realizar essa conversão em um parâmetro float32 ou nos dados de entrada, usa-se a Equação 2.6 (KRASTEVA; STOYANOV; JEKOVA, 2025):

valor_int8 = valor_f32 / fator_de_escala_f32 + ponto_zero_int8 (2.6)

De modo similar, um valor pode ser desquantizado usando a Equação 2.7 (KRASTEVA; STOYANOV; JEKOVA, 2025), por exemplo, para a saída de um modelo quantizado:

valor_f32 = (valor_int8 − ponto_zero_int8) · fator_de_escala_f32 (2.7)

Os valores de ponto_zero e fator_de_escala, conhecidos como parâmetros de quantização, podem ser determinados usando diferentes estratégias, e diferentes partes do modelo podem ser quantizadas. Em todos os casos, um conjunto de parâmetros de quantização é compartilhado por um tensor. Por isso, esses parâmetros são escolhidos de modo que o intervalo de valores que os elementos do tensor podem assumir seja mapeado para todo o intervalo [−128, 127], visando maximizar a precisão. Com quantização de faixa dinâmica (dynamic range), os pesos são quantizados para int8, com seus fatores de escala e pontos zero pré-computados armazenados como metadados, e, sem conhecer o intervalo que as ativações podem assumir, estas são quantizadas dinamicamente em tempo de execução com base em seus valores durante a inferência. Outra estratégia, que reduz ainda mais o custo em tempo de execução, é a quantização inteira completa (full integer). Nela, fornece-se ao motor de quantização um conjunto representativo de entradas possíveis; o motor executa inferência, avalia os intervalos típicos das ativações em cada camada e computa parâmetros de quantização adequados, permitindo que toda a fase de inferência seja computada apenas com aritmética inteira, sem conversões dinâmicas (ORA■S■AN; SEICULESCU; CALEANU, 2022).

2.4 Busca de Arquiteturas Neurais (NAS)

Busca de Arquiteturas Neurais (Neural Architecture Search — NAS) é o processo de buscar automaticamente arquiteturas de redes neurais, com o objetivo de otimizar uma ou mais métricas-alvo, sendo acurácia um alvo comum. Abordagens para definir o espaço de busca incluem estratégias por camada (layerwise), por célula (cellwise) ou baseadas em árvore (tree-based). Arquiteturas candidatas são propostas e precisam ser avaliadas na métrica-alvo. Essa avaliação pode envolver treinar o modelo completo ou usar uma métrica proxy mais rápida, como treinar por menos épocas (TAN et al., 2019) ou treinar em um problema similar, porém mais simples (ZOPH et al., 2018). Após avaliar uma arquitetura candidata, há diferentes formas de explorar o espaço de busca para encontrar a arquitetura ótima, como algoritmos evolutivos, aprendizado por reforço e métodos guiados por gradiente (SAHA; SANDHA; SRIVASTAVA, 2022).

Para aplicações em nuvem, a métrica-alvo a otimizar frequentemente é apenas acurácia, ou uma combinação de acurácia e latência. Nesse cenário, memória e armazenamento são abundantes e

dificilmente são uma preocupação. Entretanto, em TinyML o cenário muda, e NAS torna-se crucial também para minimizar o uso de ROM e RAM dos modelos. Frameworks de NAS “convencionais” não são, portanto, adequados para TinyML, pois sua busca costuma ser configurada para encontrar modelos grandes demais (PEREGO et al., 2022). Por isso, são necessários frameworks de NAS voltados a TinyML, como (LIBERIS; DUDZIAK; LANE, 2021), (TAN et al., 2019) e (BURRELLO et al., 2023).

2.5 Benchmarking em TinyML

Para atender às demandas de desenvolvedores de TinyML, fornecedores frequentemente lançam novos produtos de software e hardware, como frameworks com código otimizado ou microcontroladores que prometem menor consumo ou inferência mais rápida para modelos de IA. Contudo, sem modelos padronizados e métodos de medição acordados, torna-se difícil fazer comparações justas entre produtos. Padrões de benchmarking existem para resolver esse problema e, nesta seção, apresentamos o que é considerado o conjunto de benchmarking de fato para TinyML, o MLPerf Tiny (BANBURY et al., 2021).

2.5.1 MLPerf Tiny

O MLPerf Tiny visa melhorar benchmarks existentes como o CoreMark(r) da EEMBC (GAL-ON; LEVY, 2012), que não representa adequadamente cargas reais de ML, e também o MLMark(r) da EEMBC (TORELLI; BANGALE, 2021) e o MLPerf (REDDI et al., 2020), que, embora considerem cargas de ML, não oferecem modelos pequenos o suficiente para caber em plataformas “tiny”. Outro objetivo do MLPerf Tiny é ser flexível, não assumindo métodos de implantação nem frameworks de TinyML, para suportar a diversidade desse ambiente.

A metodologia de medição definida pelo benchmark consiste em 5 lotes (batches) de, no mínimo, 10 inferências ou 10 segundos de inferências, fazendo a média dos tempos de inferência. O mesmo processo é definido para medições de energia, que exigem sensores adicionais. Um detalhe importante é que ele mede apenas a chamada de inferência, não incluindo pré ou pós-processamento como quantização, desquantização e normalização das entradas. Um desenvolvedor que deseja fazer benchmark de um dispositivo precisa implementar funções para leitura e escrita via UART (frequentemente usada para comunicação com o PC hospedeiro), bem como funções para temporização da inferência e para inicialização do modelo e execução de inferência, no framework de TinyML escolhido. Uma implementação de referência dessas funções é fornecida para uma placa STM32 e usando TensorFlow Lite for Microcontrollers como framework (BANBURY et al., 2021).

Além da metodologia de medição, a padronização mais importante do MLPerf Tiny são os modelos de benchmark. Eles definem quatro modelos padrão que resolvem tarefas típicas de TinyML: Classificação de Imagens, Visual Wake Words, Keyword Spotting e Detecção de Anomalias. A arquitetura de cada modelo é brevemente explicada abaixo.

2.5.1.1 Modelo de Classificação de Imagens

O modelo de Classificação de Imagens é uma versão de um ResNet (HE et al., 2016) com três blocos. Cada bloco consiste em uma série de camadas de convolução e de batch normalization, com uma conexão residual que “pula” cada bloco e se conecta ao final. O modelo termina com uma camada de pooling, seguida por um classificador totalmente conectado com ativação softmax, indicando a probabilidade de cada uma das 10 classes de imagem.

2.5.1.2 Modelo de Visual Wake Words

O modelo de Visual Wake Words consiste em múltiplos blocos alternando camadas de convolução e de convolução em profundidade, com batch normalization após cada uma. Após cada bloco, o número de canais aumenta e a largura e altura das características diminuem. O modelo termina com pooling e um classificador totalmente conectado com ativação softmax, indicando a probabilidade de haver ou não uma pessoa na imagem.

2.5.1.3 Modelo de Keyword Spotting

O modelo de Keyword Spotting inicialmente realiza uma convolução com um kernel retangular. Em seguida, alterna entre convolução em profundidade e convolução regular, todas com kernels quadrados, com batch normalization entre elas e mantendo o mesmo número de canais em toda a rede. Termina com pooling e um classificador totalmente conectado com ativação softmax, que determina as probabilidades de cada classe de palavras do espectrograma dado como entrada.

2.5.1.4 Modelo de Detecção de Anomalias

O modelo de Detecção de Anomalias possui a arquitetura mais simples entre os quatro modelos de benchmark. Trata-se de um autoencoder, isto é, ele tenta comprimir e descomprimir o sinal de entrada; quando não consegue reconstruí-lo adequadamente, isso pode indicar uma anomalia não vista no conjunto de treino. Ele realiza isso apenas com camadas totalmente conectadas, com uma camada central muito mais estreita do que as demais, onde ocorre a compressão.

2.6 Frameworks de TinyML

Para executar uma rede neural treinada em um microcontrolador, o caminho padrão é usar um framework de TinyML existente. Um framework, nesse contexto, é um conjunto de ferramentas que fornece, entre outras coisas, duas funcionalidades principais: uma forma de armazenar os parâmetros do modelo e as operações das camadas em ROM e uma biblioteca em C/C++ que implementa inicialização e inferência (SAHA; SANDHA; SRIVASTAVA, 2022).

Há vários frameworks de TinyML disponíveis, como µTVM, STM32Cube.AI, µTensor, DORY (BURRELLO et al., 2021) e TensorFlow Lite for Microcontrollers (DAVID et al., 2021). Como o TensorFlow Lite for Microcontrollers (TFLM) foi escolhido para este trabalho, nesta seção exploramos em mais detalhes como ele funciona.

2.6.1 TensorFlow Lite for Microcontrollers

O TFLM foi desenvolvido com o objetivo de ser um framework de inferência minimalista e agnóstico a hardware. Isso significa que, em nome da portabilidade, ele não depende de recursos que muitos processadores embarcados de baixo consumo não possuem, como sistemas operacionais ou integrações específicas de hardware. A biblioteca de inferência é um conjunto de arquivos C++ que pode ser compilado para praticamente qualquer plataforma, embora também forneça Makefiles para alguns núcleos populares, como a linha ARM Cortex-M. Para as ferramentas que geram o modelo em ROM, os desenvolvedores reutilizaram a estrutura madura do TensorFlow Lite, que converte um modelo pré-treinado .tflite em um array C.

Os desenvolvedores do TFLM optaram por um funcionamento baseado em interpretador, em vez de chamadas diretas às funções dos operadores. Um framework que faz chamadas sequenciais diretas se parece com o exemplo da Figura 2.6 — nesse caso, as chamadas dos operadores, bem como a inicialização correta dos pesos para cada operador, precisam ser escritas pelo desenvolvedor ou geradas por um gerador

fornecido pelo toolchain do framework. Já no caso do interpretador, o modelo armazenado em ROM contém não apenas os pesos, mas também quais operadores pertencem a cada camada, e o interpretador interpreta o modelo em tempo de execução, decidindo qual operador invocar. Isso adiciona alguma sobrecarga, mas os autores de (DAVID et al., 2021) afirmam que o tempo de execução de uma chamada de inferência é majoritariamente dominado pelos próprios operadores, e que a sobrecarga do interpretador é desprezível em comparação. Assim, a invocação de inferência no TFLM se assemelha ao exemplo da Figura 2.7, o que adiciona flexibilidade, permitindo que um modelo diferente seja usado simplesmente trocando o array do modelo carregado no interpretador, sem precisar regenerar e recompilar todo o projeto.

Outro elemento do TFLM é o OpResolver. Como os operadores a serem chamados são determinados em tempo de execução, o TFLM precisa de uma forma de saber, em tempo de compilação, quais operadores serão necessários; caso contrário, ele linkaria todas as funções da biblioteca e o binário seria muito maior do que o necessário. Para resolver isso, o TFLM exige que todos os operadores usados sejam adicionados manualmente ao objeto resolver.

Por fim, o interpretador também precisa de uma região de memória pré-alocada para usar em todas as ativações das camadas intermediárias que ele invoca. Isso é necessário porque o TFLM não depende de alocação dinâmica de memória; assim, o usuário deve escolher um tamanho para esse array, chamado tensor arena. Infelizmente, o TFLM não oferece um método para determinar o tamanho adequado da tensor arena, então cabe ao usuário testar diferentes valores. Se o tamanho da tensor arena for pequeno demais, o TFLM gera erro em tempo de execução; se for grande demais, a inferência funciona, mas mais RAM do que o necessário é alocada, o que é desperdício. Além disso, não é trivial prever quanta tensor arena será necessária para um dado modelo, porque o TFLM usa uma estratégia de otimização de memória para reutilizar regiões da tensor arena alocadas pelo interpretador, considerando o “tempo de vida” (lifetime) dos buffers intermediários.

2.6.2 STM32Cube.AI

O STM32Cube.AI é um framework de TinyML desenvolvido pela STMicroelectronics. Diferentemente do TFLM, ele é proprietário e suporta apenas placas STM. Ele está disponível via interface gráfica no STM32Cube IDE ou via interface de linha de comando, recebendo um modelo e gerando todo o projeto C/C++ que implementa as operações necessárias para executá-lo. Esse toolchain também oferece ferramentas para auxiliar o uso do TFLM, incluindo um utilitário que estima o tamanho da tensor arena.

Outra diferença importante entre STM32Cube.AI e TFLM é que o primeiro não utiliza interpretador; em vez disso, ele gera sequencialmente todo o código necessário para executar inferência. Além disso, ele gera código específico para cada microcontrolador STM32 suportado, realizando otimizações no código de inferência, na organização de memória e na configuração do microcontrolador. Isso é mostrado em (CROCIONI et al., 2020) como gerando economias significativas para placas STM32, com economias reportadas de cerca de 20% no tempo de inferência, 28% no uso de RAM e 14% no uso de ROM. Ele também não requer interferência do desenvolvedor: todo o código necessário é gerado automaticamente.